

MiniCPU em C

Grupo 3 - Fatorial

Simulação do ciclo Fetch-Decode-Execute com cálculo de 5! por somas repetidas.

1. Equipe e responsabilidades

Integrante	Responsabilidade
Arthur Reis	Implementação
Gabriel Peixoto	Testes e conferência do trace
Luísa Fischer	Apresentação
Sofia Villela	Slides

A equipe é formada por quatro integrantes. A tabela apresenta a divisão de tarefas; a validação pessoal de Gabriel será registrada no roteiro de testes após sua execução.

2. Objetivo do desafio

Carregar N=5 no endereço 0x10, calcular seu fatorial usando multiplicações por soma repetida e gravar **120 no endereço 0x20**. As instruções e os dados são carregados diretamente na memória do simulador.

3. Estrutura da MiniCPU

A memória tem 256 posições de 8 bits. Os registradores R0, R1, R2 e R3 também armazenam valores de 0 a 255. O PC começa em 0 e aponta para a próxima instrução. A flag ZF recebe 1 quando CMP compara valores iguais e 0 quando são diferentes.

Cada instrução ocupa três bytes: opcode, operando 1 e operando 2. Foram implementadas as dez operações: LOAD, STORE, ADD, SUB, MOV, CMP, JMP, JZ, JNZ e HALT.

4. Ciclo de execução

Fetch: a função fetch lê três bytes a partir do PC e soma 3 ao PC.

Decode: o switch de decode_execute identifica o opcode.

Execute: o case correspondente modifica os registradores, a memória ou o PC. Apenas CMP altera ZF.

Após cada instrução, trace mostra a operação, R0 a R3, PC, ZF e MEM[0x20]. O laço principal repete o ciclo até HALT. Cada ciclo contado representa uma instrução simulada, não uma medida de tempo do processador real.

Como o fatorial é calculado

5. Multiplicação por soma repetida

O resultado começa em 1. Para multiplicar pelo fator atual, o programa guarda o resultado anterior em 0x21, zera R0 e soma esse valor tantas vezes quanto o fator indica. Depois reduz o fator em 1 e repete o processo.

Fator	Somas realizadas	Resultado
5	1 + 1 + 1 + 1 + 1	5
4	5 + 5 + 5 + 5	20
3	20 + 20 + 20	60
2	60 + 60	120
1	120	120

R0 acumula as somas; R1 funciona como contador e depois recebe o próximo fator; R2 guarda o valor somado; R3 recebe 0 ou 1 para as comparações e os decrementos. O endereço 0x22 preserva o fator enquanto R1 é decrementado.

O laço interno vai de 0x58 a 0x6A. O salto em 0x7C retorna ao início da próxima multiplicação, em 0x4F. Quando o fator chega a zero, STORE grava R0 em 0x20 e HALT encerra a execução. Para N=0, um salto inicial vai direto ao armazenamento do resultado 1.

6. Organização da memória

Endereço	Conteúdo
0x00 a 0x02	JMP 0x40: desvia da região reservada aos dados
0x10	Entrada N=5
0x20	Resultado final
0x21 e 0x22	Resultado anterior e fator atual
0x40 a 0x84	Instruções numéricas do fatorial, incluindo HALT

Como dados e instruções compartilham a memória, colocar todo o programa a partir de 0x00 causaria sobreposição com 0x10 e 0x20. O salto inicial preserva o PC inicial exigido e mantém essas regiões separadas.

7. Limites da solução

A implementação usa apenas C e sua biblioteca padrão. Não usa biblioteca de emulação, multiplicação para calcular o fatorial ou função pronta de fatorial. A conversão para uint8_t mantém ADD e SUB no intervalo de 8 bits.

O fatorial exato cabe em um byte para N entre 0 e 5. Para N=6, o resultado armazenado é 208 (720 módulo 256). O programa também verifica instruções inválidas, leituras fora da memória e um limite de 10.000 ciclos.

Execução e validação

8. Como executar

Com GCC instalado, abra o terminal na pasta do projeto e compile:

```
gcc -std=c11 -Wall -Wextra -Wpedantic main.c -o minicpu
```

No Windows, execute `.\minicpu.exe`. No Linux/macOS, execute `./minicpu`. O arquivo `trace.txt` contém a execução padrão completa.

9. Resultado verificado

Na verificação técnica do projeto, a compilação com avisos tratados como erros foi concluída sem avisos. Para $N=5$, a execução terminou após **158 ciclos**, com $R0=120$, $PC=133$, $ZF=1$ e $MEM[0x20]=120$.

Ciclo	Instrução	R0	PC	ZF	MEM[0x20]
1	JMP 0x40	0	64	0	0
157	STORE R0, 0x20	120	130	1	120
158	HALT	120	133	1	120

O PC final vale 133 porque a busca do HALT, localizado em 130 (0x82), já avançou três posições. O `trace` mostra o estado depois da instrução.

N	Resultado esperado	Resultado observado
0	1	1
1	1	1
2	2	2
3	6	6
4	24	24
5	120	120

Também foram conferidos os dez opcodes, os saltos condicionais nos dois casos, a atualização de ZF, a aritmética de 8 bits e o exemplo `MOV + MOV + ADD + HALT` do enunciado. O programa e a entrada permaneceram preservados durante o cálculo.

Teste da equipe: Gabriel deve repetir os testes e registrar os resultados no arquivo `GUIA_EQUIPE.md`. As verificações acima não são apresentadas como testes já realizados por ele.

10. Referências e repositório

Material do professor: `CPU_Atividade.pdf`, arquitetura MiniCPU, ciclo de instrução e desafio do Grupo 3 (seções 3 a 6). Conceitos de busca, decodificação e execução: [Intel, The Journey Inside - Microprocessors](#), lição 2.

GitHub do projeto:

<https://github.com/afreis-dev/minicpu-grupo-3>